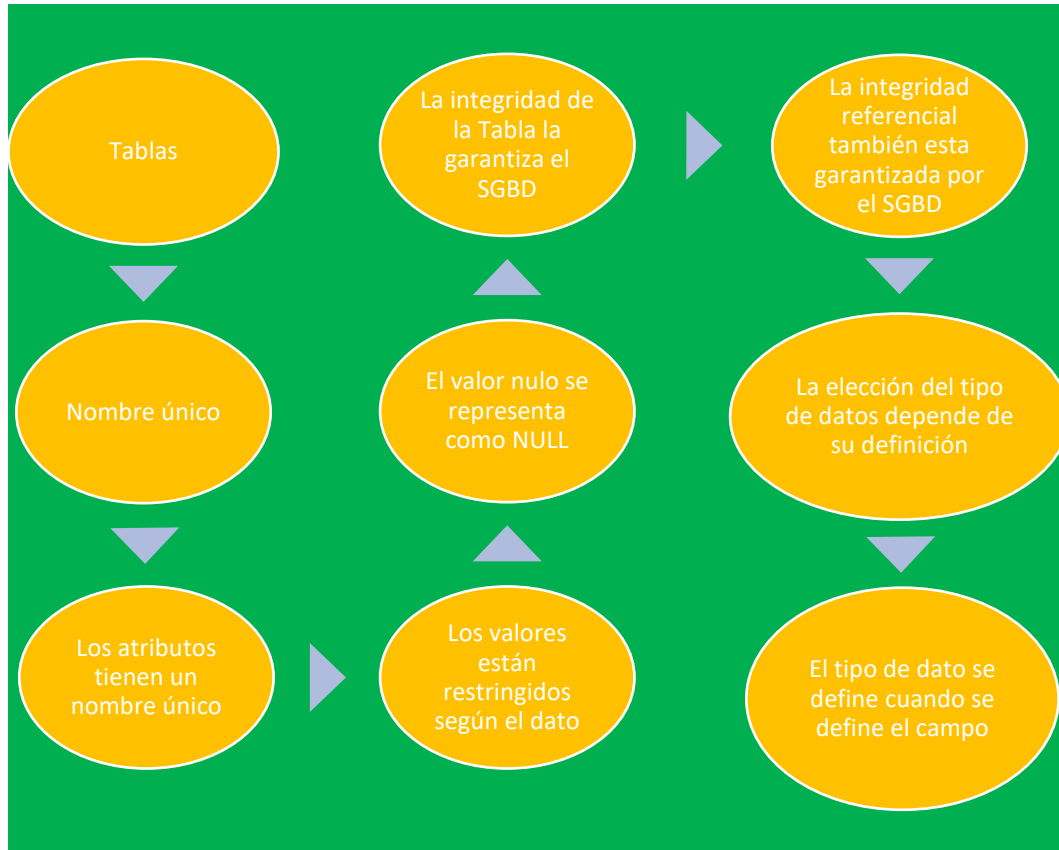


Unidad II: Contenido Adicional.

Tipos de Datos

En Base de Datos no usamos un lenguaje de programación, sino un lenguaje declarativo, que incluye expresiones, sentencias y funciones, por ejemplo.

Dentro del Modelo Relacional, se deben cumplir determinadas condiciones:



Números exactos

INTEGER

Sirven para almacenar **números enteros exactos**. La mayoría de las claves primarias usan alguno de estos tipos.

Un entero normal (**INT o INTEGER**) almacena de -2147483648 a 2147483647. Sin signo, el rango va de 0 a 4294967295

Pueden tener signo (**signed**) o no (**unsigned**), en tal caso los valores negativos se hacen positivos.

Dentro de los datos **INTEGER**, podemos enumerar **TINYINT**, **SMALLINT**, **MEDIUMINT**, **BIGINT**.

DECIMAL Y NUMERIC:

Sirven para almacenar **datos numéricos exactos con dígitos decimales**, por ejemplo, datos monetarios. Para estudios científicos que requieran mayor precisión numérica se utiliza **FLOAT y DOUBLE**, estudios bacteriológicos, etc.

- datos 4 bytes para precisión simple → **FLOAT**
- datos 8 bytes para precisión doble → **DOUBLE**

Al declarar una columna **DECIMAL**, indicada para **datos monetarios**, se debe indicar la cantidad total de dígitos almacenados enteros, así como los dígitos almacenados después de la coma o decimales.

CADENAS:

- **CHAR** se define indicando entre paréntesis, el tamaño en cantidad de caracteres que se quieren almacenar. Si el espacio asignado no es suficiente, los caracteres serán truncados.

El tamaño es fijo, ya sea que se use o no. Se autocompletará con caracteres los espacios a la derecha no usados.

El tamaño máximo es de 255 caracteres.

- **VARCHAR**: se define indicando entre paréntesis, el tamaño en cantidad de caracteres que se quieren almacenar.

El tamaño es variable, no se autocompletará con caracteres el espacio en blanco.

Si la cadena es más larga, se trunca según el tamaño declarado.

El tamaño máximo está limitado solo por el máximo tamaño de un registro, que por defecto es 65535 bytes.

- **TEXT**: se utilizan para almacenar cadenas de caracteres grandes

- TINYTEXT
- TEXT
- MEDIUMTEXT
- LONGTEXT

FECHA Y HORA

1. DATE:

- a. Solo se utiliza para datos de fecha
- b. Usa el formato **YYYY-MM-DD**
- c. Rango de valores desde 1000-01-01 hasta 9999-12-31

2. DATETIME:

- a. Se utiliza para datos que incluyen fecha y horas
- b. Usa el formato **YYYY-MM-DD hh:mm:ss**
- c. Rango de valores desde 1000-01-01 00:00:00 hasta 9999-12-31 23:59:59

3. TIMESTAMP:

- a. La diferencia con DATETIME es que la fecha se almacena como UTC
- b. Rango de valores posibles 1970-01-01 00:00:01 UTC a 2038-01-19 03:14:07 UTC

CREAR UNA BASE DE DATOS

Usamos la siguiente sentencia:

```
CREATE DATABASE nombre_nueva_bd;
```

O podemos usar

```
CREATE SCHEMA nombre_nueva_bd;
```

Estamos creando una base de datos vacía. En esa base de datos vamos a crear tablas para ir completando el modelo.

CREAR UNA TABLA

La sintaxis básica es:

```
CREATE TABLE nombre_tabla (
    definiciones
) ENGINE=InnoDB;
```

ENGINE=InnoDB → especifica que la tabla usará el motor de almacenamiento **InnoDB**, que es uno de los motores más potentes y usados en **MySQL**.

InnoDB → ofrece

- **Soporte** para transacciones (**BEGIN, COMMIT, ROLLBACK**)
- **Integridad referencial** (llaves foráneas con **ON DELETE** y **ON UPDATE**)
- **Bloqueo a nivel de fila**, lo cual mejora el rendimiento de operaciones concurrentes
- **Recuperación automática ante fallos**.

CORRECCIÓN DE ERRORES DE PLANTEO DE InstituciónEducativa

- **Tabla Alumnos**

Actualmente	Corrección
idInstitucion VARCHAR(25) ,	idInstitucion INT , FOREIGN KEY (idInstitucion) REFERENCES InstitucionEducativa(idInstitucionEducativa)

- Tabla Clase → error tipográfico

Actualmente	Corrección
FOREIGN KEY (idAdministrativo) REFERENCES PersonalAdministrativo(idPerso nalAdministativo)ok	FOREIGN KEY (idAdministrativo) REFERENCES PersonalAdministrativo (idPersonalAdministrati vo)

- Tabla Docente → campo CUIL tiene que tener validación de 11 dígitos

Corrección
CUIL CHAR(11) UNIQUE CHECK (CUIL REGEXP '^[0-9]{11}\$'),

- Tabla Clase → columna duplicada
- Validación campo email

Corrección
Email VARCHAR(50) NOT NULL UNIQUE

OPCIONES PARA CORREGIR ERRORES

A) Rehacer las tablas

1. Eliminar la table con errores

```
DROP TABLE Clase;
```

2. Volver a crearla de manera correcta

```
CREATE TABLE Clase (  
  idClase INT PRIMARY KEY AUTO_INCREMENT,  
  Tipo ENUM ('Presencial', 'Virtual', 'Híbrida'),  
  Materia VARCHAR(50),  
  Fecha DATE,  
  Horainicio TIME,  
  HoraFin TIME,  
  idDocente INT,  
  idAdministrativo INT,  
  FOREIGN KEY (idDocente) REFERENCES  
  Docente(idDocente),  
  FOREIGN KEY (idAdministrativo) REFERENCES  
  PersonalAdministrativo(idPersonalAdministrativo)  
);
```

3. Ventaja → limpieza del código
4. Desventaja → perder los datos si ya se habían cargado
5. **Advertencia** → verificar el funcionamiento de la base de datos antes de cargar los datos

B) Corregir los errores de las tablas con ALTER TABLE

1. Si la columna o atributo esta mal creada o relacionada:
 - a. Eliminar la FK
 - b. Corregir la columna o atributo
 - c. Volver a cargar la FK correctamente
 - d. Ejemplo

-- 1. Eliminar la foreign key rota (si existe)

```
ALTER TABLE Clase DROP FOREIGN KEY  
fkAdministrativo;
```

-- 2. Corregir el nombre de la columna si está mal

-- Supongamos que la columna estaba mal escrita
como 'idPersonalAdministrativo'

```
ALTER TABLE PersonalAdministrativo  
CHANGE COLUMN idPersonalAdministrativo  
idPersonalAdministrativo INT;
```

-- 3. Volver a crear la foreign key

```
ALTER TABLE Clase  
ADD CONSTRAINT fkAdministrativo  
FOREIGN KEY (idAdministrativo) REFERENCES  
PersonalAdministrativo(idPersonalAdministrativo);
```

2. Ventaja → no se pierden los datos, cuando ya estaban siendo cargados

3. Desventaja → es trabajoso y hay que tener cuidado en no alterar el orden, ni saltarse los pasos.

Manos a la Obra

PASO 1: eliminar tabla con errores

```
DROP TABLE IF EXISTS ClaseAlumno;  
DROP TABLE IF EXISTS Clase;
```

PASO 2: crear tabla Clase correctamente

```
CREATE TABLE Clase (  
  idClase INT PRIMARY KEY AUTO_INCREMENT,  
  Tipo ENUM ('Presencial', 'Virtual', 'Híbrida'),  
  Materia VARCHAR(50),  
  Fecha DATE,  
  HoraInicio TIME,  
  HoraFin TIME,  
  idDocente INT,  
  idAdministrativo INT,  
  FOREIGN KEY (idDocente) REFERENCES Docente(idDocente),  
  FOREIGN KEY (idAdministrativo) REFERENCES PersonalAdministrativo(idPersonalAdministrativo)  
);
```

PASO 3: crear la tabla ClaseAlumno de manera correcta

```
CREATE TABLE ClaseAlumno (  
  idClaseAlumno INT PRIMARY KEY AUTO_INCREMENT,  
  idClase INT,  
  idAlumnos INT,  
  MaterialEntregado BOOLEAN,  
  Observaciones TEXT,  
  FOREIGN KEY (idClase) REFERENCES Clase(idClase),  
  FOREIGN KEY (idAlumnos) REFERENCES Alumnos(idAlumnos)  
);
```

ANALICEMOS EL RESULTADO

Tabla	Relaciones actuales (con claves foráneas)
Administracion	X No se relaciona con nadie (tabla colgada)
PersonalAdministrativo	☒ Usada en Clase.idAdministrativo
InstitucionEducativa	X No se relaciona con nadie (colgada)
PadreTutor	☒ Usada en Alumnos.idPadreTutor, PagoAlumno.idPadreTutor
Alumnos	☒ Relacionada con PadreTutor, ClaseAlumno, Asistencia
Docente	☒ Relacionado con Clase, PagoDocente
Clase	☒ Relacionada con Docente, PersonalAdministrativo, ClaseAlumno, Asistencia
ClaseAlumno	☒ Usa Clase y Alumnos
Asistencia	☒ Usa Clase y Alumnos
PagoAlumno	☒ Usa PadreTutor
PagoDocente	☒ Usa Docente

Tablas colgadas detectadas

1. Administracion

Está creada, pero nadie hace referencia a ella.

Posibles soluciones:

- Eliminarla si no se va a usar.
- Relacionarla, con alguna tabla que corresponda según su función, por ejemplo con InstitucionEducativa, Clase, o PersonalAdministrativo.

2. InstitucionEducativa

En la tabla Alumnos hay una columna idInstitucion, pero está como VARCHAR(25) cuando debería ser INT y clave foránea.

Reparación del código:

```
ALTER TABLE Alumnos
MODIFY COLUMN idInstitucion INT,
ADD FOREIGN KEY (idInstitucion) REFERENCES
InstitucionEducativa(idInstitucionEducativa);
```

SCRIPT DE CORRECCIÓN COMPLETO

Antes de comenzar las modificaciones→

Guardamos una copia de seguridad

1. Abrí tu consola de MySQL o Workbench.
2. Exportá tu base de datos actual:
3. Si estás usando MySQL Workbench: **File > Export > Export to Self-Contained File.**

Ejecutamos el script por partes

4. Primero corremos hasta el **DROP TABLE IF EXISTS....** Y verificamos que se borre bien ☒
5. Luego probamos por tabla, verificando que no de error
6. Cuando todas estén listas, generamos en Database, en Ingeniería Inversa el DER

Si algo sale mal, como tenemos un respaldo, volvemos al estado anterior.

Entonces, vamos con el código modificado:

-- Seleccionamos la base de datos

USE InstitucionEducativa;

-- Paso 1: Eliminar tablas que tienen dependencias primero (en orden inverso)

DROP TABLE IF EXISTS Asistencia;

DROP TABLE IF EXISTS ClaseAlumno;

DROP TABLE IF EXISTS Clase;

DROP TABLE IF EXISTS PagoAlumno;

DROP TABLE IF EXISTS PagoDocente;

DROP TABLE IF EXISTS Alumnos;

DROP TABLE IF EXISTS Docente;

DROP TABLE IF EXISTS PadreTutor;

DROP TABLE IF EXISTS PersonalAdministrativo;

DROP TABLE IF EXISTS Administracion;

DROP TABLE IF EXISTS InstitucionEducativa;

-- Paso 2: Crear tablas corregidas y relacionadas

CREATE TABLE InstitucionEducativa (

idInstitucionEducativa INT PRIMARY KEY,

Nombre VARCHAR(25),

Dirección VARCHAR(75)

);

CREATE TABLE Administracion (

id INT PRIMARY KEY,

Nombre VARCHAR(25),

Apellido VARCHAR(25),

Email VARCHAR(25) NOT NULL,

idInstitucionEducativa INT,

**FOREIGN KEY (idInstitucionEducativa) REFERENCES
InstitucionEducativa(idInstitucionEducativa)**

);

CREATE TABLE PersonalAdministrativo (

idPersonalAdministrativo INT PRIMARY KEY AUTO_INCREMENT,

Nombre VARCHAR(25),

Apellido VARCHAR(25),

Email VARCHAR(25) NOT NULL

CHECK (Email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$'),

Telefono VARCHAR(25)

);

```
CREATE TABLE PadreTutor (  
    idPadreTutor INT PRIMARY KEY AUTO_INCREMENT,  
    Nombre VARCHAR(25),  
    Apellido VARCHAR(25),  
    Dirección VARCHAR(75),  
    Teléfono VARCHAR(25),  
    FormaPago VARCHAR(25),  
    CondicionPermanencia ENUM ('Activo', 'Completado', 'Cesante') NOT  
    NULL  
);
```

```
CREATE TABLE Alumnos (  
    idAlumnos INT PRIMARY KEY AUTO_INCREMENT,  
    Nombre VARCHAR(25),  
    Apellido VARCHAR(25),  
    Dirección VARCHAR(75),  
    Teléfono VARCHAR(25),  
    Mail VARCHAR(25),  
    Año INT,  
    idInstitucion INT,  
    idPadreTutor INT,  
    FOREIGN KEY (idInstitucion) REFERENCES  
    InstitucionEducativa(idInstitucionEducativa),  
    FOREIGN KEY (idPadreTutor) REFERENCES PadreTutor(idPadreTutor)  
);
```

```
CREATE TABLE Docente (  
    idDocente INT PRIMARY KEY AUTO_INCREMENT,  
    Nombre VARCHAR(25),  
    Apellido VARCHAR(25),
```

Dirección VARCHAR(75),
Teléfono VARCHAR(25),
Mail VARCHAR(25),
CUIL VARCHAR(11),
AreaIncumbencia VARCHAR(50),
HorarioDisponible VARCHAR(50),
UltimoPago DATE
);

CREATE TABLE Clase (
idClase INT PRIMARY KEY AUTO_INCREMENT,
Tipo ENUM ('Presencial', 'Virtual', 'Híbrida'),
Materia VARCHAR(50),
Fecha DATE,
HoraInicio TIME,
HoraFin TIME,
idDocente INT,
idAdministrativo INT,
FOREIGN KEY (idDocente) REFERENCES Docente(idDocente),
FOREIGN KEY (idAdministrativo) REFERENCES
PersonalAdministrativo(idPersonalAdministrativo)
);

CREATE TABLE ClaseAlumno (
idClaseAlumno INT PRIMARY KEY AUTO_INCREMENT,
idClase INT,
idAlumnos INT,
MaterialEntregado BOOLEAN,
Observaciones TEXT,
FOREIGN KEY (idClase) REFERENCES Clase(idClase),
FOREIGN KEY (idAlumnos) REFERENCES Alumnos(idAlumnos)

);

CREATE TABLE Asistencia (

idAsistencia INT PRIMARY KEY AUTO_INCREMENT,

idClase INT,

idAlumnos INT,

Presente BOOLEAN,

Fecha DATE,

FOREIGN KEY (idClase) REFERENCES Clase(idClase),

FOREIGN KEY (idAlumnos) REFERENCES Alumnos(idAlumnos)

);

CREATE TABLE PagoAlumno (

id INT PRIMARY KEY AUTO_INCREMENT,

idPadreTutor INT,

Fecha DATE,

Total DECIMAL(10,2),

Forma_pago VARCHAR(50),

Pago_adelantado BOOLEAN,

Detalle TEXT,

FOREIGN KEY (idPadreTutor) REFERENCES PadreTutor(idPadreTutor)

);

CREATE TABLE PagoDocente (

idPagoDocente INT PRIMARY KEY AUTO_INCREMENT,

idDocente INT,

Fecha DATE,

Cantidad_horas INT,

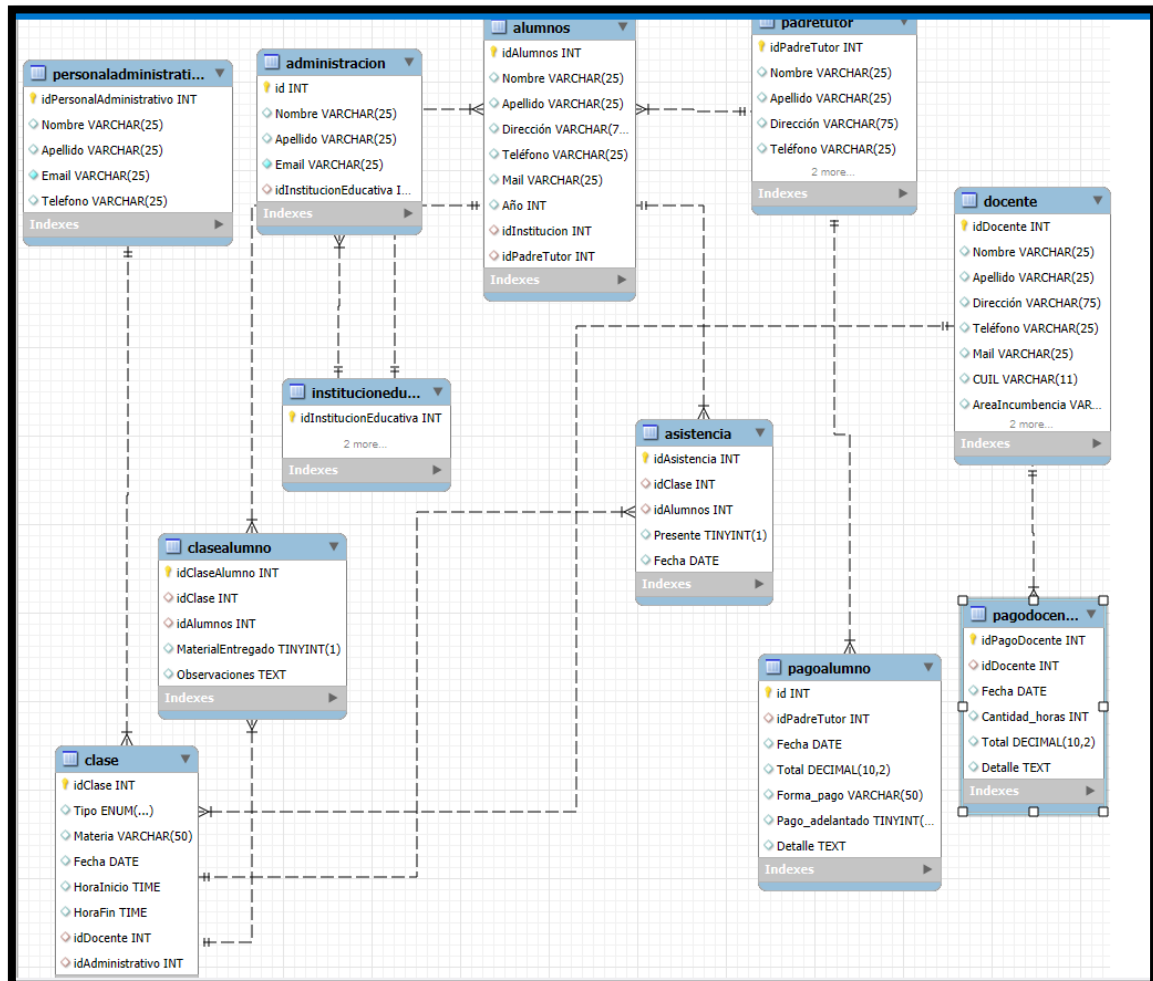
Total DECIMAL(10,2),

Detalle TEXT,

FOREIGN KEY (idDocente) REFERENCES Docente(idDocente)

);

OBTENEMOS EL DER, DESDE DATABASE, OPCIÓN REVERSE ENGINEER



SEGUIMOS MEJORANDO LA VERSIÓN

En el modelo actual la Institución no participa en el pago al docente, sino que lo hace el padre o tutor-

OBJETIVO

- El alumno (a través del tutor) paga a la Institución.
- La Institución paga al Docente.
- Los registros de pagos deben reflejar esas relaciones.

Tablas intermedias

Una tabla intermedia o puente, es la que permite representar relaciones de muchos a muchos:

- Un alumno puede tener muchas clases
- Una clase puede tener muchos alumnos

No se puede modelar directamente con FK, es necesario una tabla de vinculación.

En nuestro modelo, la **tabla intermedia es ClaseAlumno: Relaciona**

- ✓ **idClase (de la tabla Clase)**
- ✓ **idAlumnos (de la tabla Alumnos)**

El script de la tabla es:

```
CREATE TABLE ClaseAlumno (  
    idClaseAlumno INT PRIMARY KEY AUTO_INCREMENT,  
    idClase INT,  
    idAlumnos INT,  
    MaterialEntregado BOOLEAN,  
    Observaciones TEXT,  
    FOREIGN KEY (idClase) REFERENCES Clase(idClase),  
    FOREIGN KEY (idAlumnos) REFERENCES Alumnos(idAlumnos)  
);
```


Para que se vincule la **Institución** pagando al **docente**, vamos a generar otra **tabla intermedia**

Reemplazamos la tabla PagoAlumno por una tabla intermedia entre PadreTutor e InstitucionEducativa.

Entonces relacionamos PagoDocente con InstitucionEducativa, en lugar de hacerlo directamente con el docente.

Seguimos los siguientes pasos:

1. Eliminamos PagoAlumno

```
DROP TABLE IF EXISTS PagoAlumno;
```

2. Creamos table intermedia PagoPadreTutor

```
CREATE TABLE PagoPadreTutor (  
  idPagoTutor INT PRIMARY KEY AUTO_INCREMENT,  
  idPadreTutor INT,  
  idInstitucionEducativa INT,  
  Fecha DATE,  
  Total DECIMAL (10,2),  
  Forma_pago VARCHAR (50),  
  Pago_adelantado BOOLEAN,  
  Detalle TEXT,  
  FOREIGN KEY (idPadreTutor) REFERENCES  
  PadreTutor(idPadreTutor),  
  FOREIGN KEY (idInstitucionEducativa) REFERENCES  
  InstitucionEducativa(idInstitucionEducativa)  
);
```

3. Modificamos PagoDocente para vincularlo a la InstituciónEducativa, en lugar de estar directamente con el docente:

a. Eliminamos la tabla

```
DROP TABLE IF EXISTS PagoDocente;
```

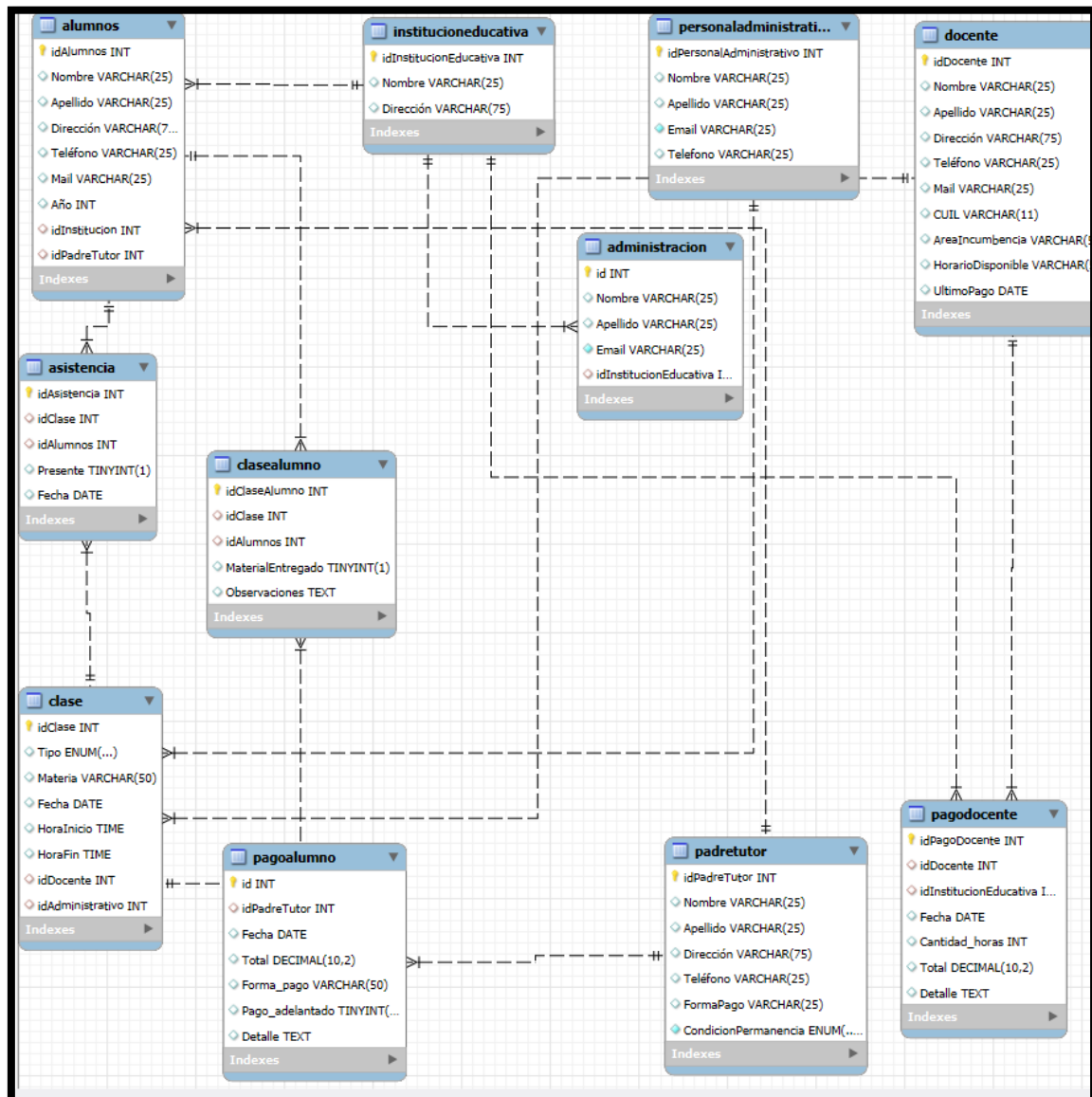
b. Creamos la relación adecuada

```
CREATE TABLE PagoDocente (  
  idPagoDocente INT PRIMARY KEY AUTO_INCREMENT,  
  idDocente INT,  
  idInstitucionEducativa INT,  
  Fecha DATE,  
  Cantidad_horas INT,  
  Total DECIMAL(10,2),  
  Detalle TEXT,  
  FOREIGN KEY (idDocente) REFERENCES  
  Docente(idDocente),  
  FOREIGN KEY (idInstitucionEducativa) REFERENCES  
  InstitucionEducativa(idInstitucionEducativa)  
);
```

Entonces ahora la **Institución es el nexo central** entre padres o tutores y los docentes que imparten las clases

Podemos visualizar en línea nuestro DER en <https://www.plantuml.com/plantuml/uml/>

Sino hacemos las modificaciones en el script de MySQL y luego generamos el DER en la pestaña Database, Reverse Engineer



AHORA VAMOS A CARGAR LOS DATOS